Calibração de Câmera (TA02)

Millena Suiani Costa, Seijiro Yanagiya Midzuno
Departamento de Informática
Universidade Federal do Paraná – UFPR
Curitiba, Brasil

https://github.com/millenaSui/VisaoComputacional-T2

I. Introdução

O presente relatório tem como objetivo apresentar a implementação prática do processo de calibração de uma câmera de celular utilizando a biblioteca **OpenCV** integrada à linguagem **Python** e imagens registradas de um tabuleiro de xadrez. O projeto foi estruturado de forma modular e conteinerizado utilizando Docker, garantindo reprodutibilidade e o isolamento do ambiente de execução.

II. Preparação das Imagens e Ambiente

Seguindo o enunciado do trabalho, que sugeria a utilização de um tabuleiro de xadrez como objeto de calibração, uniu-se a necessidade de uma superfície rígida, para garantia de qualidade na calibração, à conveniência das ferramentas dispostas para desenvolvimento do trabalho. Assim, optou-se pela utilização de um tabuleiro digital (10x10), fotografado na tela de um monitor e dispondo de borda branca (*quiet zone*) em seu entorno:

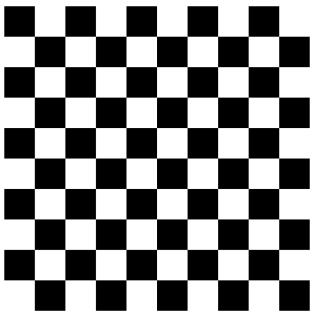


Figura 1. Tabuleiro (10x10)

A biblioteca *OpenCV* requer a declaração da quantidade de cantos internos do tabuleiro (`cv2.findChessboardCorners`). Dadas as dimensões deste, tem-se que a quantidade corresponde a 9 interseções horizontais multiplicadas por 9 interseções verticais, resultando em 81 cantos internos no total. A dimensão do tabuleiro no monitor foi mantida em 100%, de tal forma que tanto a altura quanto a largura do tabuleiro representam 250 mm da tela, ou seja, cada quadrado possui a medida exata de 25 mm. Uma vez tiradas as medidas do tabuleiro, foram registradas 15 imagens suas em várias angulações, alterando os eixos X, Y e Z. O processamento dessas imagens foi automatizado através de um pipeline organizado em módulos, onde o script principal realiza o carregamento a partir da pasta `images/dataset`.

III. Extração de Características e Calibração

Para dar início ao processo de calibração, as imagens capturadas foram convertidas para escala de cinza (*grayscale*), simplificando a detecção das formas geométricas do tabuleiro e a localização das coordenadas 2D dos 81 cantos internos nas imagens. Visando maior precisão, aplicou-se um refinamento através da função `cv2.cornerSubPix`, que ajusta iterativamente a posição dos cantos detectados para uma precisão além dos limites do pixel inteiro, adotando como critério de parada 30 iterações ou um epsilon de 0.001.

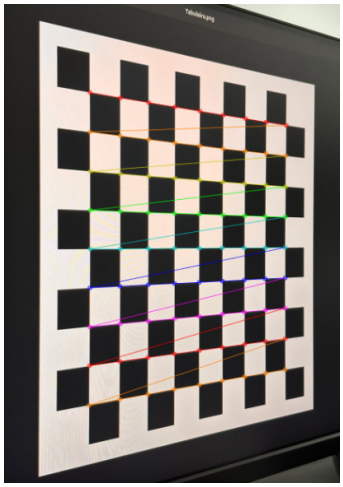


Figura 2. Identificação de Cantos Internos

As coordenadas 3D dos pontos no espaço (no caso, assumindo Z=0 por se tratar de um objeto plano) foram geradas com base nas dimensões de 25 mm medidas anteriormente. Com os conjuntos de pontos 2D (nas imagens) e 3D (no espaço) devidamente estabelecidos, utilizou-se a função `cv2.calibrateCamera`. Esta função recebe os pontos e a resolução das imagens para retornar a **matriz intrínseca da câmera**, os **coeficientes de distorção**, além dos vetores de rotação e translação referentes à posição da câmera em cada imagem, assim como o erro de reprojeção.

IV. Remoção da Distorção e Verificação

Uma das aplicações práticas da calibração é a correção de imperfeições nas lentes da câmera, notadamente as distorções radiais (efeito "barril" ou "almofada") e tangenciais. Utilizando os coeficientes de distorção e a matriz da câmera obtidos, o algoritmo calculou primeiramente uma matriz otimizada com o auxílio da função `cv2.getOptimalNewCameraMatrix`. Com esta nova matriz, a função `cv2.undistort` foi aplicada a cada uma das imagens originais.

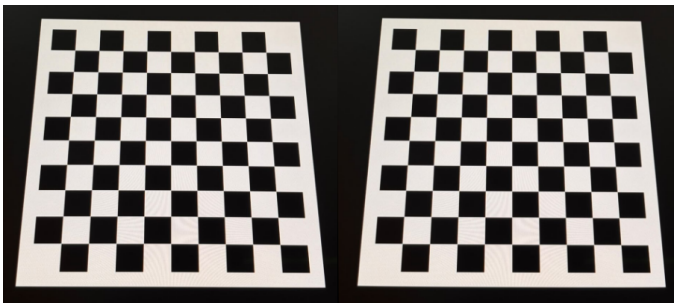


Figura 3. Comparativo (Antes x Depois)

Para validar a qualidade da calibração realizada, conduziu-se um experimento de reprojeção. Este processo consiste em utilizar a matriz da câmera e os vetores de posicionamento (rotação e translação) para projetar os pontos 3D ideais do tabuleiro de volta no plano 2D da imagem através da função `cv2.projectPoints`.

Em seguida, as distâncias Euclidianas entre os pontos reais detectados na imagem e as posições teóricas projetadas foram calculadas para encontrar o erro médio quadrático em pixels. Nas imagens validadas, os pontos detectados originais foram marcados com círculos vermelhos e os pontos projetados com cruzes verdes. Erros pequenos (na ordem de frações de pixel ou poucos pixels) indicam uma calibração bem-sucedida, atestando que os parâmetros intrínsecos e extrínsecos modelam de forma fidedigna a câmera utilizada no experimento.

V. O Resultado

A diferença visual entre as fotos originais e as fotos corrigidas pelo algoritmo se fez bastante sutil e isso se deve ao fato de, ao contrário de câmeras analógicas antigas ou webcams simples, os smartphones atuais utilizam processadores de sinal e imagem para aplicar uma série de correções para o melhor resultado possível da foto final, incluindo a remoção natural das distorções da própria lente. Ou seja, quando as fotos do *dataset* foram tiradas, já haviam passado por um processo de tratamento no próprio celular.

Por conta disso, quando foram calculados os **coeficientes de distorção** da câmera utilizada, valores muito baixos foram encontrados, visto que, analisando a imagem de forma matemática, identificou-se que havia pouquíssima distorção residual a ser tratada e, assim, foram feitos apenas os ajustes mínimos necessários, mantendo a imagem resultante muito parecida visualmente com a original.